

El uso de herramientas de inteligencia artificial no está prohibido en esta actividad. Sin embargo, cada punto o ejercicio deberá resolverse en un chat o hilo independiente dentro de la herramienta utilizada, por ejemplo: ChatGPT, Gemini, DeepSeek, entre otras.

Al finalizar cada punto, se deberá adjuntar el enlace al chat correspondiente como evidencia del proceso realizado. En caso de que la herramienta de inteligencia artificial que se desee emplear no permita compartir el hilo o chat utilizado, su uso no estará permitido para esta actividad.

Asimismo, se deberá crear un repositorio de GitHub independiente por cada punto del examen. En cada repositorio deberán estar disponibles todos los archivos, códigos, comandos, configuraciones y demás recursos empleados para el desarrollo del punto correspondiente.

Cada captura de pantalla debe mostrar la hora. En el caso de las capturas de AWS se debe poder observar el nombre del usuario.

1. Utilizando el proyecto `lambda_final` basado en FastAPI, realice el despliegue en Lambda cumpliendo con los siguientes pasos:
 - a. Cree el archivo Dockerfile necesario para empaquetar la aplicación dentro de un contenedor Docker compatible con AWS Lambda.
 - b. Desarrolle un script en Bash que:
 - i. Construya la imagen de Docker.
 - ii. Asigne la etiqueta `latest` a la imagen.
 - iii. Suba la imagen al container registry de AWS
 - c. Ejecútelo y haga una captura de pantalla donde se observe la ejecución del archivo y en otra captura la imagen publicada correctamente dentro del registry.
 - d. Cree una función Lambda utilizando la imagen previamente almacenada en el registry.
 - e. Configure una Lambda Function URL para exponer la aplicación y permitir pruebas externas del servicio. Tome captura de pantalla de esta configuración.
 - f. Muestre una captura de pantalla evidenciando el uso del comando `curl` para consumir el endpoint de la API y verificar su funcionamiento correcto.
2. Desarrolle y despliegue en una instancia EC2 una aplicación FastAPI que cumpla lo siguiente:
 - a. Implementar un endpoint POST que reciba:
 - El nombre de un usuario.
 - Una imagen en formato PNG o JPG/JPEG.

- b. Validaciones requeridas:
 - Validar que únicamente se acepten archivos en los formatos permitidos.
 - En caso de recibir un formato inválido, retornar un código HTTP de error del lado del cliente (por ejemplo 400 o 415).
- c. El endpoint debe:
 - Almacenar la imagen en un bucket S3, organizando los archivos por usuario.
- d. Implementar un endpoint GET que reciba:
 - Nombre de usuario.
 - Nombre de la imagen.
- e. El endpoint debe:
 - Verificar la existencia tanto del usuario como de la imagen solicitada dentro del bucket S3.
 - En caso de no existir, retornar un mensaje claro indicando el problema.
 - Retornar una URL válida para acceder a la imagen almacenada, por ejemplo, una URL prefirmada.
 - Incluir en la respuesta la fecha de almacenamiento obtenida desde los metadatos del objeto en S3
- f. Validar localmente ambos endpoints utilizando Swagger y tomar capturas de pantalla como evidencia (no utilizar curl).
- g. Desplegar la aplicación en una instancia EC2 y habilitar el acceso público a la API. Tomar diferentes capturas donde se observe el proceso realizado
- h. Crear el archivo de servicio de systemd (.service) necesario para ejecutar la aplicación FastAPI como servicio dentro de la instancia EC2. No olvidar subir este también a GitHub.
- i. Ingresar al Swagger de la instancia pública, probar ambos endpoints y tomar capturas de pantalla como evidencia del funcionamiento.